

突破字节前端2-1面试： JS异步编程问题应答范式及进阶

一、前言

本期让我们来聚焦Javascript异步

异步编程在现代软件开发中至关重要，它解决了阻塞和资源利用率两大核心问题，从根本上提升了程序的效率、响应能力和可扩展性

JavaScript的异步无论在前端还是Node服务、工程化领域，都是基础中十分重要的组成部分。异步相关的问题，在面试中也是重点高频问题

思考以下几个问题，看看你对JavsSript异步编程了解的程度有多少？

- 如果让你把线程、进程、Event Loop、SetTimeout、Ajax、 ReadFile、Promise、Generator、Async Await这些关键词分组，你会如何分？ 他们之间的关系是什么？
- 你对Promise的理解是什么？ Promise的核心解决了什么问题？
- 你了解genrator自执行器吗？ 它解决了前段异步编程中的什么问题？
- Async await的本质是什么？
- 上次公开课的request的高阶答法，如何设计http client, 可以自动识别调用的地方是否进行了错误处理？

二、Event Loop

【重点掌握】 JavaScript异步基础 - 事件循环执行过程

事件循环是JavaScript实现异步的基础，与其他多线程的编程语言的实现不一致，因JavaScript单线程的特性，导致异步的实现需要基于Event Loop

我们先看一段代码：

Code block

```
1  console.log('Sync Code');
2
3  setTimeout(() => {
4      console.log('Async Code');
5  }, 2000);
6
7  console.log('Sync Code');
```

这段代码是怎么执行的呢？

- 首先JavaScript的解释器在读到 `console.log('Sync Code')` 的时候， 控制台输出了 `'Sync Code'`
- 之后解释器调用`setTimeout()`， 设置了一个定时器，在2000ms后执行 `console.log('Async Code');`
- 再次读到 `console.log('Sync Code')` ,控制台输出 `'Sync Code'`

- 所有同步代码执行完毕，进入Event Loop过程
- 两秒后向控制台输出console.log('Async Code');

同步执行与异步执行

同步执行的代码包含初始执行代码和异步创建的同步代码两种

异步执行的代码通过一些内置的**API**创建，常见的异步代码创建API有：

- 双端（浏览器、NodeJS）API及语法：

- setTimeout
- setInterval
- Promise 相关API

- 浏览器端API：

- MutationObserve
- XMLHttpRequest
- requestAnimationFrame
- queueMicrotask

- Node端API：

- nextTick
- ReadField等等

JavaScript的运行机制即为，执行完所有的同步代码后，进入Event Loop

让我们观察用于创建异步执行代码的API，可以发现，这些API的参数一定是一个回调函数

Code block

```
1  setTimeout(() => {
2  })
3
4  nextTick(() => {})
5
6  new Promise().then(() => {
7  })
8  ...
```

这些被包裹的待执行函数，会被推入一个等待执行的队列，Event Loop即为一个无限的循环过程，去检查队列中是否存在等待执行的函数。如果存在，将其压入调用栈执行，在JavaScript执行线程存在的生命周期中一直持续这个过程

伪代码如下

Code block

```
1  const PendingQueue = []
2  exec(SyncCode);
3
4  while(true) {
5      if (PendingQueue.length) {
6          exec(PendingQueue.shift());
7      }
8  }
```

【重点掌握】宏任务微任务

宏任务与微任务

JavaScript将待执行栈分为宏任务与微任务。区别在于对于宏任务与微任务的执行的优先级不同

执行优先级

Views

?

Enter code to show diagram

宏任务与微任务的创建API

所有创建异步的API，仅有少数的API可用于创建微任务，其余均是宏任务，记住这几个微任务即可

创建微任务的API与语法：

- Promise实例相关API
- nextTick
- queueMicrotask

【特殊】

requestAnimationFrame 是浏览器端特有的异步过程的创建API，其执行时机位于微任务和宏任务之间，并且还受限于浏览器是否有渲染任务（涉及到帧率）

三、Promise

【重点掌握】为什么会有Promise

1. 控制权反转问题

所谓的控制权反转，即为在早期以callback（回调函数）来实现异步执行时，需要把异步执行函数做为回调传入到创建异步的函数中

Code block

```
1  setTimeout(() => {
2    console.log('async');
3  }, 1000);
```

如果是运行环境中的官方的API函数还好，但是如果是一个第三方封装的函数，将无法确定传入的回调函数会被如何执行

Code block

```
1  // 例如有一个第三方的异步支持函数
2  const asyncPay = (payCallBack) => {
3      setTimeout(() => {
4          // 这里执行了三次
5          payCallBack()
6          payCallBack()
7          payCallBack()
8      }, 1000)
9  }
10
11
12 // 调用方视角
13 const pay = () => console.log('pay')
14 asyncPay(pay);
```

也可以理解为，控制权反转即把将要执行的异步过程和创建异步的函数的执行进行了耦合，为了解决这个问题，那么就需要把「执行过程调用」改变为「消息通知」，即创建异步任务的函数不再负责直接调用后续的异步过程，而是仅仅「发送」异步过程的结果，后续的调用由发起方自行控制。这即是Promise解决控制反转问题的核心。

Promise如何解决的？

通过状态机，Promise只负责告知当前的异步过程执行的状态（pending - 等待/ fulfilled - 正常/ rejected - 异常），并设计了对应的API来响应状态的变化

一个Promise的状态只可以改变一次，且只能从pending改变为fulfilled/rejected

初始化

Promise通过初始化创建来包裹异步过程，并在合适的时机修改状态机。

1. 通过构造函数中注入的resolve和reject方法

Code block

```
1  const createPromise = () => {
2      return new Promise((resolve, reject) => {
3          setTimeout(() => {
4              resolve('success')
5          }, 1000)
6      })
7  }
8
9  const p = createPromise();
10 p.then((res) => {
11     alert(res);
12 })
13
```

创建微任务

当Promise构造函数中的**resolve \ reject被执行时**，会把通过Promise实例中通过.then()传入的函数创建一个微任务。所以Promise.then创建的微任务实际是通过 resolve\reject来创建的

Code block

```
1  const createPromise = () => {
2      return new Promise((resolve, reject) => {
3          setTimeout(() => {
4              resolve('success')
5              console.log('1');
6          }, 1000)
7      })
8  }
9
10 const p = createPromise();
11 p.then((res) => {
12     console.log(res);
13     setTimeout(() => {
14         console.log(3)
15     }, 500)
16     return res;
17 }).then((res) => {
18     console.log(4)
19 })
20
21 输出：1->suceess->4->3
```

2. 回调地狱

Code block

```
1  doTask1(result1 => {
2      doTask2(result1, result2 => {
3          doTask3(result2, result3 => {
4              // 更多嵌套...
5          });
6      });
7  });
```

【重点掌握】Promise链式调用

通过Promise实例的API(then catch finally)会返回一个新的Promise实例

API接受的函数的返回值可以是任意值，Promise会判断返回值的类型并进行相应的处理

返回Promise

Code block

```
1  // 返回Promise
2  const promise = Promise.resolve().then(() => {
3      return new Promise(res => res());
4  });
5
```

```
6 console.log(promise);
7
8 // => Promise {<pending>}
```

返回thenable(函数或者对象的原型链上有then, 且then为函数): promise 把resolve传入到then的参数中

Code block

```
1 t promise = Promise.resolve().then(() => {
2     return {
3         then: (onFullfield) => {
4             onFullfield();
5         }
6     }
7 })
8
9 console.log(promise);
10
11 promise.then((res) => {
12     console.log(promise)
13 });
14
15 // Promise {<pending>}
16 // Promise {<fulfilled>}: undefined
17
```

返回其他值: 直接返回一个resolved的promise

Code block

```
1 promise = Promise.resolve().then(() => {
2     return {
3         then: 1
4     }
5 })
6
7
8 promise.then((res) => {
9     console.log(promise)
10 });
11
12 // Promise {<fulfilled>}: {then:1}
13
```

【重点掌握】Promise核心API

- `Promise.all(iterable)`: `Promise.all()` 接受一个Promise实例组成的迭代器（例如数组）作为参数，并返回一个新的Promise实例。当所有输入的Promise都resolved时，这个新的Promise才会resolved，并且返回一个包含所有输入Promise resolved值的数组。如果其中任何一个Promise被reject，`Promise.all()` 也会立即reject，并且返回的Promise会包含第一个被reject的Promise的错误信息。
- `Promise.allSettled(iterable)`: `Promise.allSettled()` 类似于 `Promise.all()`，但它会在所有输入的Promise都settle (resolved 或rejected) 后才返回。它返回一个对象数组，每个对象描述了对应的Promise的状态和结果。对于resolved的Promise，对象包含 `status: "fulfilled"` 和 `value` 属性；对于rejected的Promise，对象包含 `status: "rejected"` 和 `reason` 属性。

- `Promise.race(iterable)`: `Promise.race()` 接受一个Promise实例组成的迭代器作为参数，并返回一个新的Promise实例。只要其中任何一个输入的Promise状态发生改变（resolved 或rejected），`Promise.race()` 就会立即返回，并使用该Promise的结果作为自己的结果。如果迭代器为空，则返回的Promise将永远pending。
- `Promise.resolve(value)`: `Promise.resolve(value)` 用于创建一个已经resolved的Promise实例。如果 `value` 本身是一个Promise实例，则该方法会返回这个Promise实例；如果 `value` 是一个thenable对象（即具有 `.then()` 方法的对象），则 `Promise.resolve()` 会调用该对象的 `.then()` 方法，并根据其返回值来决定新的Promise的状态；如果 `value` 不是Promise或thenable对象，则返回一个以该值为resolved值的Promise实例。
- `Promise.reject(reason)`: `Promise.reject(reason)` 用于创建一个已经rejected的Promise实例。`reason` 是reject的原因。

四、Generator & yield

【了解】为什么会有Generator & yield

Generator & yield 本来不仅仅是为了解决异步问题，其本身还解决了迭代器的中断/继续问题

Code block

```
1  // ES5 数组迭代[1,2,3].forEach(v => console.log(v));
2  // 无法中途暂停/恢复迭代
3  // 无法外部控制迭代过程
```

所以其本身为执行过程控制流，允许终端函数的执行，并在后续某个时机（可以是同步/异步）继续执行

Generator & yield 解决了异步什么问题？

Promise解决了javascript异步编程中的控制权反转问题，但是由于其同步/异步过程的执行存在跳跃、不连贯，不符合人类的思维习惯，导致Promise存在心智负担

Code block

```
1  const createPromise = () => {
2      return new Promise((resolve, reject) => {
3          setTimeout(() => {
4              resolve('success')
5              console.log('1');
6          }, 1000)
7      })
8  }
9
10 const p = createPromise();
11 p.then((res) => {
12     console.log(res);
13     setTimeout(() => {
14         console.log(3)
15     }, 500)
16     return res;
17 }).then((res) => {
18     console.log(4)
19 })
20
21 console.log('5')
22
23 输出：5->1->suceess->4->3
```


而Generator & yield 由于其中断/继续函数执行的控制能力，让基于Promise的异步可以以一种同步的方式进行

【了解】 Generator & yield 的作用以及自迭代器的实现

Code block

```
1  const createPromise = (v) => {
2    return new Promise((resolve, reject) => {
3      setTimeout(() => {
4        resolve(v)
5      }, 1000)
6    })
7  }
8
9  // asyncFlow里面的代码输出都是同步的了，这里并不是实际的执行位置
10 function* asyncFlow() {
11   console.log(1);
12   const res = yield createPromise(2); // 暂停点1
13   console.log(res)
14   console.log(3);
15   const res2 = yield createPromise(4); // 暂停点2
16   console.log(res2)
17   console.log(5);
18
19 }
20
21 // 执行控制
22 const gen = asyncFlow();
23 gen.next().value.then(res => { // 启动
24   gen.next(res).value.then(res => {
25     gen.next(res)
26   });
27 })
```

在async flow中，我们的执行流程实际是由下面的执行器来进行，这样确保在asyncFlow中同步/异步代码的执行顺序是按代码顺序执行的

问题在于每次执行迭代都需要手动的调用gen.next，但是本身asyncFlow中需要中断的异步过程的数量未知，如何优化这个问题？

Genertor自执行器

Code block

```
1  const createPromise = (v) => {
2    return new Promise((resolve, reject) => {
3      setTimeout(() => {
4        resolve(v)
5      }, 1000)
6    })
7  }
8
9  function* asyncFlow() {
10   console.log(1);
11   const res = yield createPromise(2); // 暂停点1
12   console.log(res)
13   console.log(3);
14   const res2 = yield createPromise(4); // 暂停点2
15   console.log(res2)
```



```
16     console.log(5);
17
18 }
19
20 // 执行控制 - generator runtime
21 function runGenerator(genFunc) {
22     // 创建Generator实例
23     const generator = genFunc();
24
25     // 递归处理函数
26     function handle(result) {
27         // 如果Generator执行完成，返回最终值
28         if (result.done) {
29             return Promise.resolve(result.value);
30         }
31
32         // 确保返回的是一个Promise
33         const promise = Promise.resolve(result.value);
34
35         // 处理Promise，成功时继续下一步，失败时抛出错误
36         return promise
37             .then(data => {
38                 // 将Promise的结果传入Generator并继续执行
39                 return handle(generator.next(data));
40             })
41             .catch(err => {
42                 // 将错误抛回Generator
43                 return handle(generator.throw(err));
44             });
45     }
46     handle(generator.next());
47 }
48
49 runGenerator(asyncFlow);
```

五、Async Await

【了解】 async await的本质

本质就是Generator & yield + Generator自执行器 + Promise的语法糖。可以隐藏Generator自执行器，并且不需要显示调用AutoRun

Code block

```
1  const foo = async function() {
2      console.log(1);
3      const result = await createPromise(2);
4      console.log(result);
5      console.log(3);
6  }
```

如果await 后面不是一个Promise, 那么await会把后面的代码执行求值完毕后，通过Promise.resolve()来产生Promise对象

【了解】 async await的异常处理

async await由于从语法层面来实现异步控制，导致其在异常处理时会存在块级作用阈限制问题

Code block

```
1  const foo = async function() {
2    try {
3      const result = await createPromise(2);
4    } catch (err) {
5
6    }
7    console.log(result) // 由于异步结果在try的块级作用于里面，导致这里获取不到
8  }
```

解决方案一：

把后续同步过程放在try的块级作用阈中

Code block

```
1
2
3  const foo = async function() {
4    try {
5      const result = await createPromise(2);
6      console.log(result) // 由于异步结果在try的块级作用于里面，导致这里获取不到
7    } catch (err) {
8
9    }
10  }
11
```

问题： 会扩大、非预期catch的捕获范围（本身只捕获createPromise的错误，现在console的错误也会捕获了）

解决方案二：

返回值作用域提升：

Code block

```
1
2  const foo = async function() {
3    let result;
4    try {
5      result = await createPromise(2);
6    } catch (err) {
7
8    }
9    console.log(result) // 由于异步结果在try的块级作用于里面，导致这里获取不到
10  }
11
```

问题：

- 1. 常量编程变量，存在被修改的风险
- 2. TS不友好

解决方案三：

通过立即执行函数创建独立作用域 IIFE

Code block

```
1  const foo = async function() {
2      const result = await (async () => {
3          try {
4              return await createPromise(2); // 在IIFE内部返回结果
5          } catch (err) {
6              console.error('发生错误:', err);
7              return null; // 错误时返回默认值
8          }
9      })();
10
11     console.log(result); // 现在可以访问result
12 }
13 foo();
```

问题：

1. 冗余函数执行、样板代码

六、重点异步题目精讲

1. Promise、微任务、宏任务

Code block

```
1  console.log('script start');
2  setTimeout(() => {
3      console.log('timeout1');
4      Promise.resolve().then(() => console.log('promise1'));
5  }, 0);
6  Promise.resolve().then(() => {
7      console.log('promise2');
8
9      setTimeout(() => {
10         console.log('timeout2');
11         Promise.resolve().then(() => console.log('promise3'));
12     }, 0);
13
14     return new Promise(resolve => {
15         console.log('promise4');
16         resolve();
17     }).then(() => {
18         console.log('promise5');
19         queueMicrotask(() => console.log('microtask1'));
20     });
21 }).then(() => {
22     console.log('promise6');
23 });
24
25 new Promise(resolve => {
26     console.log('promise8');
27     resolve();
28 }).then(() => {
29     console.log('promise9');
30     return Promise.resolve().then(() => {
31         console.log('promise10');
32         setTimeout(() => console.log('timeout3'), 0);
33     });
```

```
34   }).then(() => {
35     console.log('promise11');
36   });
```

2. 实现一个自动识别catch的request client

Code block

```
1
2  type Request = (url: string) => Promise<any>
3
4  type UseRequest = (options: { limit: number }) => {
5    request: Request
6  }
7
8  // pending queue里面的runner实际上就是一个函数类型，没有参数也没有返回值
9
10 type runner = () => void;
11
12 // 这里我们mock一下异步请求
13 const mockAixos = (url: string) => {
14   const random = Math.floor(Math.random() * 1001);
15   return new Promise((res, rej) => {
16     setTimeout(() => {
17       // 看error处理的时候调用rej, 否则调用res
18       debugger
19       rej('err');
20
21       // res(`request ${url} success!`);
22     }, random)
23   })
24 }
25
26 export const useRequest: UseRequest = ({ limit = 5 }) => {
27
28   let runingCount: number = 0;
29   const pendingQueue: runner[] = [];
30
31   // 实现request内部
32   const request = (url: string) => {
33     // 由我们定义的request类型可知，我们需要返回一个promise，那么整个request首先我们需要构造一个promise实例并返回
34     // 由于100次的request请求是同步调用的，我们去做限流的时候会把暂时不能执行的请求缓存起来，所以这promise的
35     // resolve的执行时机，是调用方可以正确执行request('/test').then()的关键
36     // 构造我们的执行，并发限流处理的核心思路为我们需要设计一个执行器runner，这个runner会执行一个异步过程，同时在异步过程完成的时候，去检查我们的pendingQueue里（缓存池）面还有没有带执行的任务
37     // 如果有的话，取一个出来执行，是一个典型的泳池模型
38
39     // 这里有一个闭包内的状态来存储是否在外面调用了.catch
40     let outerCatch = false;
41     const p = new Promise((resolve, reject) => {
42       const runner = () => {
43         runingCount++
44         mockAixos(url).then((response) => {
45           // 这里是关键，这个resolve 包裹在runner里面，那么实际这个resolve是每次调用request()时候创建的promise的实例的resovle
46           // 那么如果这里延迟执行，这个resolve也会延迟执行，外面获取的promise是pending，直到runner运行时返回结果，这里才会触发resolve，那么调用的地方就可以拿到对应的response了
47           resolve(response)
48           // 同时runner运行完毕，我们需要把正在执行的runner计数器-1
```

```
49         runingCount--;
50         // 最后我们检查，在pendingQueue里面还有没有带执行的runner
51         const nextRunner = pendingQueue.shift();
52         nextRunner && nextRunner();
53     }).catch((err: string) => {
54         // 这里异常处理依据outer catch，是内部处理还是抛出
55         if (outerCatch) {
56             reject(err)
57         } else {
58             alert(`global error procee ${err}`);
59         }
60     })
61 }
62 // 首先我们先处理前5次请求，直接运行runner执行即可
63 if (runingCount < 5) {
64     runner();
65 } else {
66     // 如果正在运行的runner数量大于等于limit了，我们把runner推到pendingQueue里面
67     pendingQueue.push(runner);
68 }
69 })
70 // 通过proxy代理返回的promise，
71 // 因为promise是链式返回新的promise对象
72 // 所以我们这里需要递归的对promise的then\catch\fanally等api的返回结果再次进行代理
73
74 const proxyPromise = (targetPromise) => {
75     return new Proxy(targetPromise, {
76         get(target: any, key: string) {
77
78             // 这里如果是调用catch，则设置outerCatch为true，表示在外面调用了catch
79             if (key === 'catch') {
80                 outerCatch = true;
81             }
82
83             // 其他情况下，所有的都返回再次代理的新promise对象
84             return (cb: any) => {
85
86                 // 关键点1：这里已经是proxy的对象了，所以一定要把api都bind回原本的promise再执行
87                 const p = target[key].bind(target)(cb);
88
89                 // 关键点2：返回新的代理的promise
90                 return proxyPromise(p);
91             };
92         }
93     })
94 }
95 return proxyPromise(p);
96 }
97 // 首先实现我们hooks的返回值，即request方法
98 return {
99     request,
100 }
101
102 }
103
```

